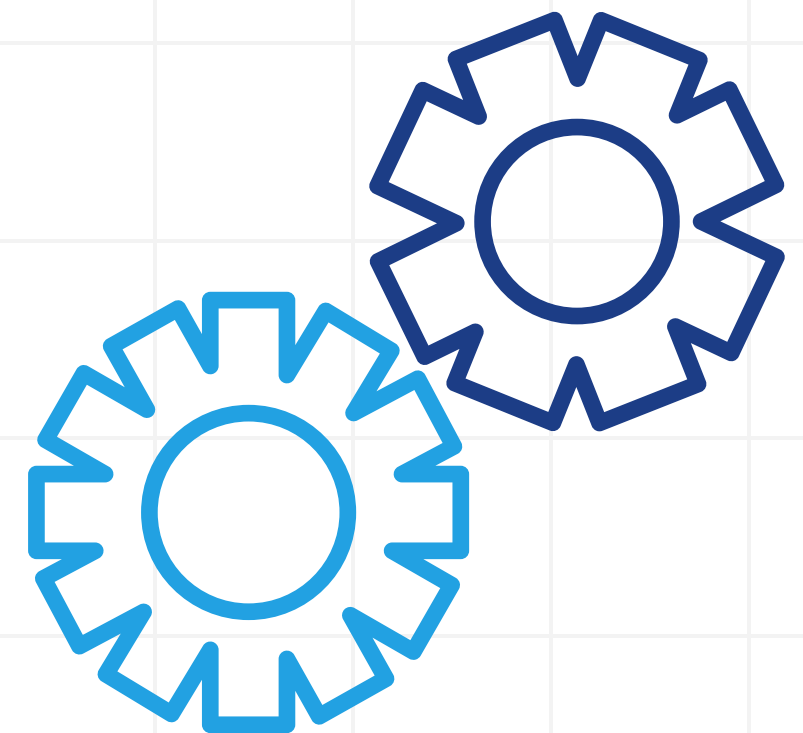




DDoS Attack Prediction Using Machine Learning

Group 3 :

- **Jonathan Christian Wongso**
- **Kenneth Maximillian Pranata**
- **Sisilia Annabel Aileen**





Motivation

- **DDoS attacks are becoming more frequent and sophisticated.**
- **Early detection can reduce service downtime, financial losses, and security risks.**
- **Machine learning can learn traffic patterns and detect attacks more effectively than static rule-based methods.**
- **The project aims to provide real-time attack detection and monitoring through a user-friendly dashboard.**





Problem Statement

- **DDoS attacks can disrupt online services by flooding networks with malicious traffic.**
- **Traditional rule-based detection systems struggle to identify new and evolving attack patterns.**
- **The large volume of network traffic makes manual monitoring ineffective for real-time threat detection.**
- **An automated and intelligent detection system is needed to identify DDoS attacks quickly and accurately.**





Dataset Description

Dataset Name : BCCC–cPacket–Cloud–DDoS–2024

Author Introduction about the dataset :

The 'BCCC–cPacket–Cloud–DDoS–2024' dataset collaboratively with cPacket from the States to address challenges identified in previous datasets through a cloud infrastructure. The dataset contains over eight benign user activities and 17 DDoS attack scenarios. The dataset is fully labeled (with a total of 26 labels) with over 300 features extracted from the network and transport layers of the traffic flows using NTLFlowLyzer.

Paper Reference : <https://www.mdpi.com/2078-2489/15/4/195>





Exploratory Data Analysis



Dataset Shape

Shape: (540494, 318)

Missing Values

Missing values total: 0

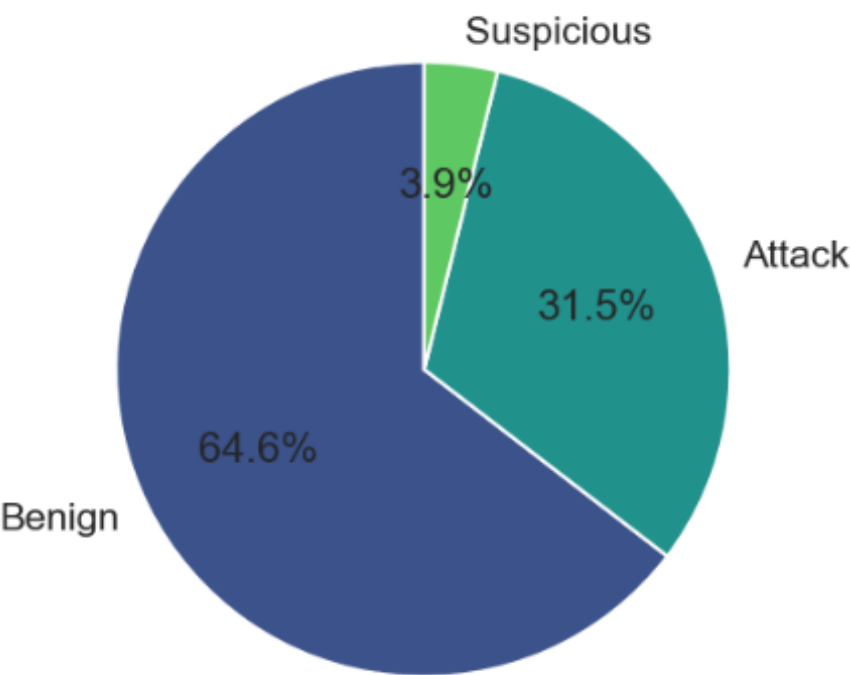
Duplicated Rows

Duplicate rows: 21391

Dataset Info

RangeIndex: 540494 entries, 0 to 540493
Columns: 319 entries, src_port to activity
dtypes: float32(261), int16(9), int32(20), int64(2), int8(25), str(2)
memory usage: 627.3 MB

Class Distribution



Dataset Summary

rows	540494
columns	318
numeric_features	317
categorical_features	0
missing_cells	0
duplicate_rows	21391
classes	3





ML Problem Formulation

- 1. How can machine learning techniques be utilized to detect DDoS attacks from network traffic data?**
- 2. Which machine learning algorithm provides the best performance for DDoS attack detection in terms of accuracy, precision, recall, F1-score, and ROC-AUC?**
- 3. How effectively can the proposed system classify network traffic into benign and DDoS categories in a near real-time monitoring environment?**
- 4. How can a machine learning-based intrusion detection system be integrated into a real-time monitoring dashboard to provide timely attack alerts and historical traffic analysis?**





Model Selection & Rationale *

1. Logistic Regression (floor)

- Simplest model to set the minimum acceptable accuracy
- Any model that can't beat it is considered a failure
- Creates a benchmark ladder: simple → moderate → complex

2. SVM (baseline)

- Represents a different algorithm family (margin-based) outside of tree-based methods
- Used to measure whether tree-based models genuinely outperform non-ensemble approaches

3. KNN (baseline)

- Represents instance-based, non-parametric learning
- Tests how distance-based methods degrade under high-dimensional network features — expected to underperform, strengthening the case for primary models

4. Random Forest (primary)

- Handles high-dimensional features natively
- Captures non-linear relationships in traffic patterns
- Robust against noisy data through tree averaging
- Optimizable for speed — fits the "Live Listen" near real-time requirement

5. XGBoost (primary)

- All Random Forest strengths, plus gradient boosting for higher accuracy
- Recognized as the best-in-class for tabular data
- Built-in regularization prevents overfitting across varied attack types
- Supports parallel processing for lightweight, fast deployment





Training & Validation Setup

- **Dataset split ratio 80% train : 10 % validation : 10% testing**
- **Imputation: Safely handles missing data**
- **Scaling dataset after split**
- **Dimensionality Reduction (Curse of Dimensionality) → PCA**
- **Hyperparameter tuning Cross-Validation**
- **Handling class imbalance → implementing `class_weight="balanced"` and `"balanced_subsample"`**
- **Tuning Metric → `"f1_macro"` finding hyperparameter for predicting all classes equally**
- **implemented multi-dimensional evaluation (F1-Score, accuracy, precision, accuracy, recall, ROC-AUC score)**





Evaluation Metric

- 1. Accuracy**
- 2. Precision**
- 3. Recall**
- 4. F1-Score**
- 5. F1-Macro vs F1-Weighted**
- 6. ROC-AUC**

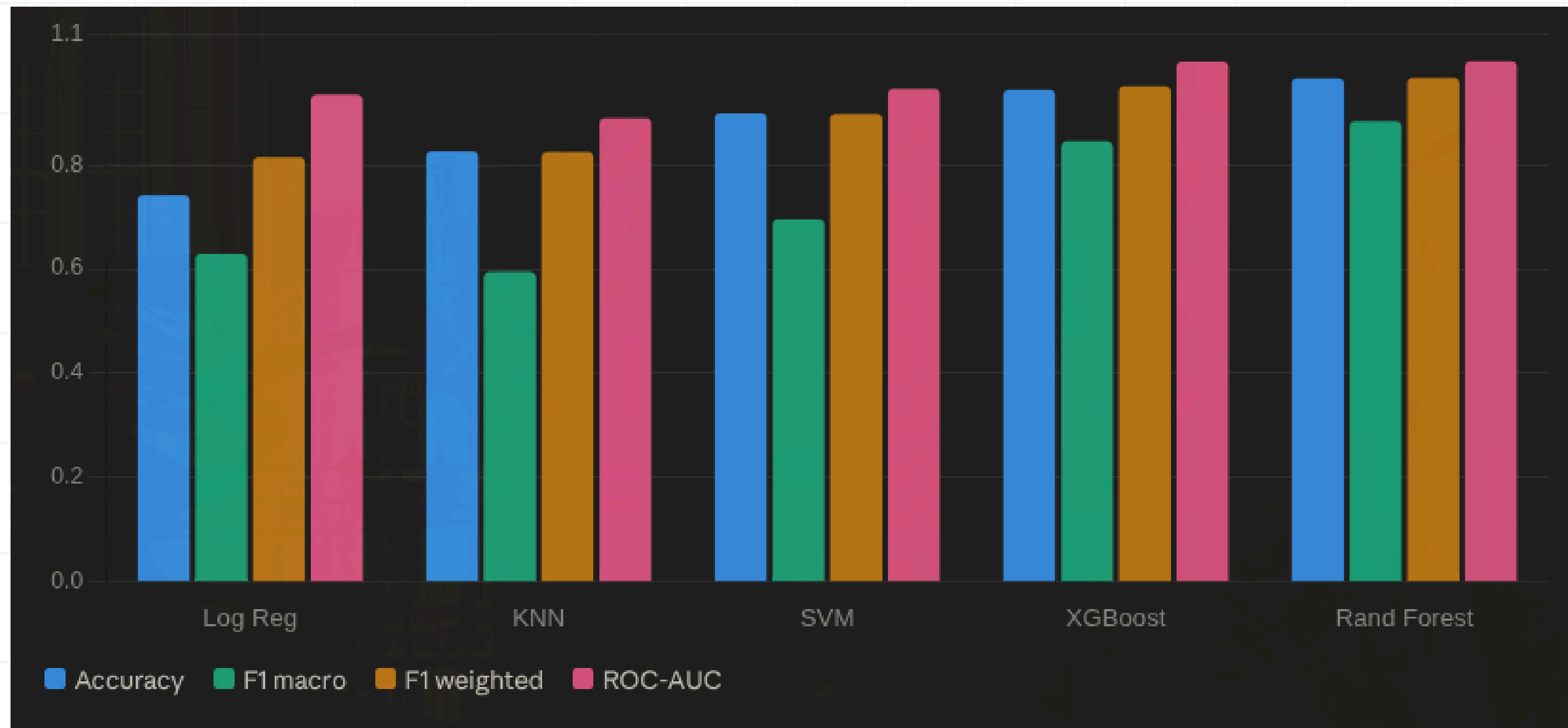




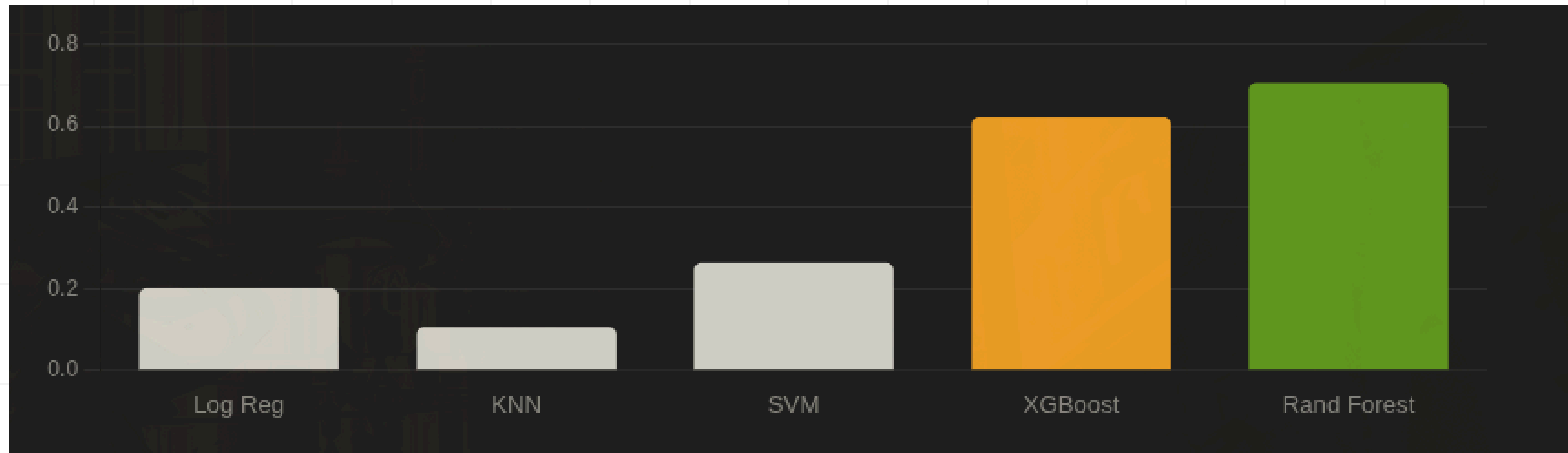
Result



Overall performance

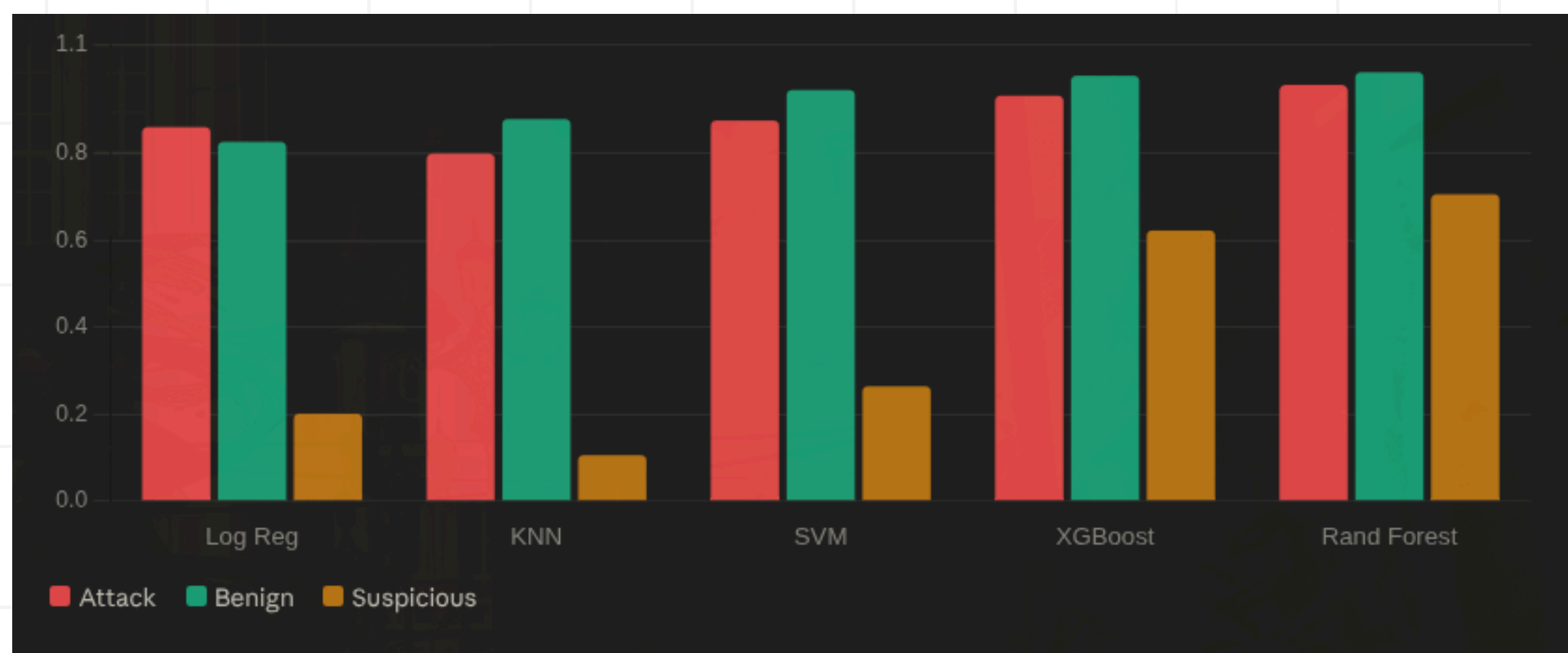


Per-class F1 score (Suspicious) ^{ooo}



Per-class F1 – all classes

○○○○



ROC-AUC comparison

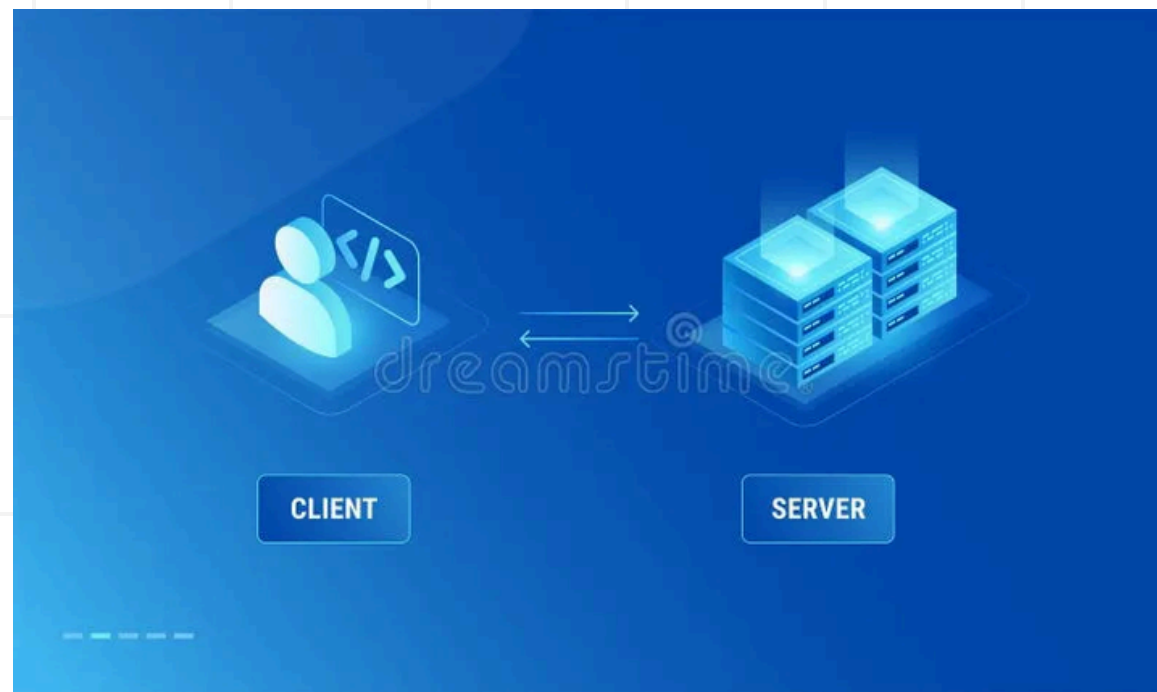




Deployment Architecture



Client–Server Architecture



The Frontend (Client Tier)

- **Role:** Acts as the **User Interface (UI)**.
- **Action:** It captures the user's input, sends those inputs via an **HTTP request** to the backend, and visually displays the returned prediction and data to the user.

The Backend (Server & Inference Tier)

- **Role:** Acts as the brain and the **API**.
- it loads the saved model file. When it receives a request from the frontend, it processes the data, runs the `predict()` function, and sends the result back to the frontend as a **JSON response**.

 **FastAPI** **Hugging Face****NEXT**.JS

Application



☒	ROW_INDEX	DURATION	PACKETS_COUNT	BYTES_RATE	PACKETS_RATE	FWD_PACKETS_RATE	BWD_PACKETS_RATE	BWD_BYTES_RATE	DOWN_UP_RATE	HANDSHAKE_ST
☒	0	1	5	745000	5000	2000	3000	582000	3.5706	0
☒	1	2049	3	96.6325	1.4641	0.488	0.9761	64.4217	2	0
☒	2	1	1	66000	1000	0	1000	66000	66	0
☒	3	1	1	66000	1000	0	1000	66000	66	0
☒	4	15360	5	21.4844	0.3255	0.1302	0.1953	12.8906	1.5	0
☒	5	6003	9	213.7265	1.4993	0.6663	0.8329	105.4473	0.9738	0
☒	6	5883	5	99.0991	0.8499	0.34	0.5099	76.6616	3.4167	0
☒	7	1	1	66000	1000	0	1000	66000	66	0
☒	8	1	1	66000	1000	0	1000	66000	66	0
☒	9	1	1	66000	1000	0	1000	66000	66	0

Page 1 of 9 · 82 total rows

... Prev Next ...

Run Prediction

Model: xgboost Predict 10 Rows

Prediction Results												
Prediction time: 61.43 ms												
PREDICTED LABEL	CONFIDENCE	ID	EXPIRATION_ID	SRC_IP	SRC_MAC	SRC_OUI	SRC_PORT	DST_IP	DST_MAC	DST_OUI	DST_PORT	PROTOCOL
Benign	0.9647	0	0	127.0.0.1	00:00:00:00:00:00	00:00:00	34466	127.0.0.1	00:00:00:00:00:00	00:00:00	37755	6
Benign	0.9744	1	0	127.0.0.1	00:00:00:00:00:00	00:00:00	53518	127.0.0.1	00:00:00:00:00:00	00:00:00	37755	6
Benign	0.9902	2	0	127.0.0.1	00:00:00:00:00:00	00:00:00	51498	127.0.0.1	00:00:00:00:00:00	00:00:00	37535	6
Benign	0.975	3	0	127.0.0.1	00:00:00:00:00:00	00:00:00	37755	127.0.0.1	00:00:00:00:00:00	00:00:00	34466	6
Benign	0.938	4	0	127.0.0.1	00:00:00:00:00:00	00:00:00	37755	127.0.0.1	00:00:00:00:00:00	00:00:00	53528	6
Benign	0.9447	5	0	127.0.0.1	00:00:00:00:00:00	00:00:00	36432	127.0.0.1	00:00:00:00:00:00	00:00:00	37535	6
Benign	0.9745	6	0	127.0.0.1	00:00:00:00:00:00	00:00:00	40251	127.0.0.1	00:00:00:00:00:00	00:00:00	58610	6
Benign	0.992	7	0	127.0.0.1	00:00:00:00:00:00	00:00:00	42568	127.0.0.1	00:00:00:00:00:00	00:00:00	37535	6
Benign	0.9822	8	0	127.0.0.1	00:00:00:00:00:00	00:00:00	40506	127.0.0.1	00:00:00:00:00:00	00:00:00	37535	6
Benign	0.9829	9	0	127.0.0.1	00:00:00:00:00:00	00:00:00	37755	127.0.0.1	00:00:00:00:00:00	00:00:00	53518	6



User Testing

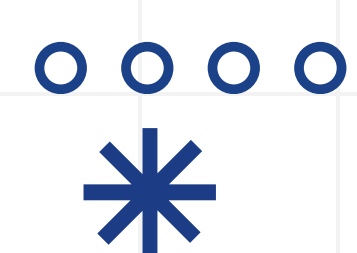


User Testing Design

- **Likert scale to measure the Usability and Usefulness from the proposed application.**
- **Ask the user on their feed back on the application**

User Testing Result

The user testing results demonstrate strong overall performance with an average score of 4.3/5 for both usability and usefulness, highlighting the system's high responsiveness, stable backend, and intuitive design for both technical and non-technical users. However, constructive feedback indicates minor areas for improvement, specifically regarding color contrast for attack indicators and the potential addition of a data export feature





Analysis





Baseline model



Logistic Regression

performance floor at 74.19% accuracy and 0.6295 macro F1.
Interesting Suspicious class recall of 80.03%, high with terrible precision (11.51%) indicates over-predicting the Suspicious cause inability to draw non-linear decision boundaries.
Its ROC-AUC of 0.9347 is surprisingly competitive, showing it can rank samples decently even if it can't classify them precisely.





The Challengers



KNN

KNN's results macro F1 of 0.5949 and Suspicious class F1 of 0.1056 (the worst), breaks down under high-dimensional, imbalanced data.

Distance calculations unreliable when features number in the dozens, and the minority Suspicious class gets effectively ignored.

Its ROC-AUC of 0.8892 is also the lowest, making it unsuitable as anything more than a comparison reference.

SVM

SVM achieves 89.91% accuracy and Attack precision of 98.11%, good labeling attack.

However, its Suspicious class F1 of only 0.2643 is a significant weakness.

The macro F1 of 0.6953 confirms SVM's margin-based approach struggles under class imbalance.

It is also worth noting its ROC-AUC of 0.9453 is far below both primary models.





Champions



XGBoost

XGBoost trails Random Forest slightly on accuracy (94.34%) and weighted F1 (95.07%), but it stands out in one critical area — the Suspicious class recall reaches 97.94%, the highest of any model. This means XGBoost almost never misses a suspicious packet, at the cost of lower precision (45.66%). If the deployment priority is minimizing false negatives (i.e., never letting a suspicious event slip through), XGBoost is the better choice despite the tradeoff.

Random Forest

Random Forest is the clear winner across almost every metric. It achieves the highest accuracy (96.57%), F1 weighted (96.67%), F1 macro (88.35%), and ROC–AUC (99.83%). Notably, it also handles the Suspicious class the best among all models in terms of balanced F1 (70.61%), making it the most production–ready candidate. Its performance validates the initial hypothesis that tree–based ensemble methods are best suited for this task.





Limitations

- **Dataset Imbalance:** The "Suspicious" traffic class makes up only 3.9% of the dataset, compared to 64.6% for Benign and 31.5% for Attack. This lack of representation makes it inherently difficult for any algorithm to perfectly classify edge-case anomalies
- **The Precision vs. Recall Trade-off:** This means the system might be overly cautious, occasionally generating "False Positives" by flagging normal network spikes as suspicious. Even our champion model, Random Forest, peaked at a 70.61% balanced F1 score for this specific minority class.





Conclusion

1. Model Performance Ranking

- **Random Forest (Champion):** Balanced F1-score of 70.61%.
- **XGBoost (High Recall Alternative):** It rarely misses a threat but generates more false positives (45.66% Precision).

2. Why Tree-Based Models Won

- **Linear models (Logistic Regression) and distance-based models (KNN) performed poorly (Macro F1 of 62.95% and 59.49%, respectively). This proves that the network traffic data contains complex, non-linear relationships and high-dimensional features that only ensemble methods like Random Forest and XGBoost can effectively navigate.**





Future Work

- **Data Rebalancing:** Apply advanced sampling techniques (e.g., SMOTE) or collect more real-world traffic logs to better train the model on the minority "Suspicious" class (currently only 3.9%)
- **Feature Expansion:** Implement a data export feature allowing cybersecurity analysts to download prediction logs in PDF/CSV format for post-incident reporting.





Appendix

Random Forest – Full Classification Report

- **Attack: Precision: 0.9676 | Recall: 0.9479 | F1-Score: 0.9576**
- **Benign: Precision: 0.9878 | Recall: 0.9857 | F1-Score: 0.9867**
- **Suspicious: Precision: 0.6465 | Recall: 0.7778 | F1-Score: 0.7061**
- **Global Metrics: * Accuracy: 0.9657**
- **Macro F1: 0.8835**
- **ROC-AUC: 0.9983**

